

# Lab-1



## 图形绘制技术

Graphic Rendering Technology

Lab1

2026 春季学期

# 目录

|                        |           |
|------------------------|-----------|
| <b>1 作业提交</b>          | <b>3</b>  |
| 1.1 分数占比               | 3         |
| 1.2 你需要编写的部分           | 3         |
| 1.3 提交方式               | 3         |
| <b>2 光线-物体求交接口</b>     | <b>3</b>  |
| 2.1 为什么需要求交            | 3         |
| 2.2 moer-lite 中的物体求交接口 | 3         |
| <b>3 圆环与光线的求交</b>      | <b>4</b>  |
| 3.1 moer-lite 中的圆环     | 4         |
| 3.2 求交                 | 5         |
| 3.3 填充求交信息             | 6         |
| 3.4 验证你的求交结果           | 6         |
| <b>4 圆柱与光线的求交</b>      | <b>6</b>  |
| 4.1 moer-lite 中的圆柱     | 6         |
| 4.2 求交                 | 7         |
| 4.3 填充求交信息             | 8         |
| 4.4 验证你的求交结果           | 8         |
| <b>5 圆锥与光线的求交</b>      | <b>8</b>  |
| 5.1 moer-lite 中的圆锥     | 8         |
| 5.2 求交                 | 9         |
| 5.3 填充求交信息             | 10        |
| 5.4 验证你的求交结果           | 11        |
| <b>6 加速结构介绍</b>        | <b>11</b> |
| 6.1 加速结构简介             | 11        |
| 6.2 BoundingBox        | 12        |
| 6.3 加速结构接口             | 13        |
| <b>7 Octree (选做)</b>   | <b>14</b> |
| 7.1 Octree 介绍          | 14        |
| 7.2 Octree 节点结构        | 15        |
| 7.3 Octree 构建          | 15        |
| 7.4 Octree 求交          | 15        |
| 7.5 验证                 | 16        |
| <b>8 BVH (选做)</b>      | <b>16</b> |
| 8.1 BVH 介绍             | 16        |

|              |    |
|--------------|----|
| 目录           | 2  |
| 8.2 BVH 节点结构 | 16 |
| 8.3 BVH 构建   | 17 |
| 8.4 BVH 求交   | 18 |
| 8.5 验证       | 18 |
| 9 景深         | 19 |
| 9.1 实现透镜相机   | 20 |
| 10 勘误 & 补充   | 23 |

# Abstract

该文档是南京大学计算机系 2026 春季学期《图形绘制技术》课程实验 1 的实验手册。

Lab1 的内容主要是完成三种物体（圆环，圆柱，圆锥）的求交判断和交点信息填充。学有余力的同学可以完成加速结构。我们提供了两种加速结构的介绍，完成一种即可获得 bonous。

各位同学如果在 lab1 实验过程中有任何疑问或想法，包括但不限于

- 实验框架的出现的 Bug
- 手册中叙述不完善的部分
- 对实验安排的建议
- 更好的框架设计

主讲老师：过洁 Email : guojie@nju.edu.cn

欢迎联系助教，帮助我们完善课程的实验部分。TAs :

- 周辰熙 QQ : 1305845549 Email : chenxizhou@smail.nju.edu.cn
- 俞贤皓 QQ : 2943003

同时，课程的框架代码以及实验文档也会不断更新改进，请同学们关注助教发布的相关信息。

# 1. 作业提交

## 1.1 分数占比

圆环求交: 30%, 圆柱求交: 30%, 圆锥求交: 30%, 薄透镜相机: 10%. 完成任意一种加速结构, bonus 20%

## 1.2 你需要编写的部分

- Disk.cpp、Cylinder.cpp、Cone.cpp 文件下的 todo 方法
- 如果你要完成加速结构: bvh.cpp 或 octree.cpp 文件下的 todo 方法。
- 实现薄透镜相机请完成 ThinLens.cpp 中的 todo

## 1.3 提交方式

请在[智汇南雍平台](#)上以 PDF 格式提交你的作业，要求包括：

- 对于物体求交部分，给出简单的求交逻辑介绍和实验结果图片
- 对于选做部分，给出加速结构实现思路以及求交时间和线性求交的对比
- 薄透镜相机的景深效果，以及实现逻辑
- 对实验的建议和吐槽

# 2. 光线-物体求交接口

## 2.1 为什么需要求交

渲染器需要模拟光线在场景中的传播过程，这需要计算光线和物体的求交。比如我们生成了一条摄像机光线，我们需要知道这条光线是否击中了物体，如果击中物体，击中的第一个物体是什么物体。如果要对物体表面进行正确的着色，我们还需要击中点的其他信息，比如法线，纹理坐标等等。

## 2.2 moer-lite 中的物体求交接口

在 moer-lite 中，物体和光线的求交接口主要有两个。

第一个函数是判断给定光线，光线是否和物体相交。如果相交，会填充 primID(图元 Id), uv 坐标。primID 在我们本次作业中都填充 0 即可。

第二个函数是通过 primid, uv 坐标等来填充相交信息。因为光线可能会先与较



远的物体计算相交，所以我们不将这两个函数合二为一来避免没有意义的填充调用在这次作业中，我们只需要填充相交点的法线，相交点的位置即可。

```
1 virtual bool rayIntersectShape(Ray &ray, int *primID, float *u, float *v) const = 0;
2 virtual void fillIntersection(float distance, int primID, float u, float v, Intersection *
   intersection) const = 0;
3
```

在给定光线后，场景中的加速结构承担光线和场景的求交，逻辑如下：

```
1 std::optional<Intersection> rayIntersect(Ray &ray) const {
2     int primID, geomID = -1;
3     float u, v;
4     bool hit = rayIntersect(ray, &geomID, &primID, &u, &v);
5     if (!hit)
6         return std::nullopt;
7     Intersection its;
8     shapes[geomID]->fillIntersection(ray.tFar, primID, u, v, &its);
9     return std::make_optional(its);
10 }
```

其中 rayIntersect 是加速结构需要实现的方法，暂时可以认为是遍历每个物体，调用物体的 rayIntersectShape 方法。场景与光线的求交逻辑便显而易见：如果没有物体相交就返回 std::nullopt 代表没有相交物体。相交的话得到最近相交物体的 id。根据 id 找到对应物体，根据 primId 和 uv 坐标等来填充相交信息结构体。下面让我们来完成三种新增物体的求交逻辑实现吧！

## 3. 圆环与光线的求交

### 3.1 moer-lite 中的圆环

在三维空间，圆可以用圆心，法线，半径来定义。如果要计算光线和圆的交点，可以通过计算光线与平面的交点并计算交点与圆心的距离来完成。但是三维空间内的旋转等是件很麻烦的事情。我们不妨将圆固定在原点，并且让它始终垂直于 z 轴。它的旋转平移等变换通过变换矩阵来完成。在 transform 类中，我们提供了 inverseRay 方法。该方法可以将光线变换到局部空间。已经实现的物体求交中，cube 便是通过这种方法完成求交的。本次实验的圆柱，圆锥等也是通过先将光线变换到物体局部空间进行求交。

在有了以上信息，要确定一个圆其实就只需要一个半径了。为了能表示圆环，我们加入了 innerRadius 表示内半径。为了能够渲染残缺圆环，我们加入 phiMax 来表示圆环的最大角度（弧度制）

```
1 class Disk{
2     float radius;
```

```

3 float phiMax;
4 float innerRadius;
5 }

```

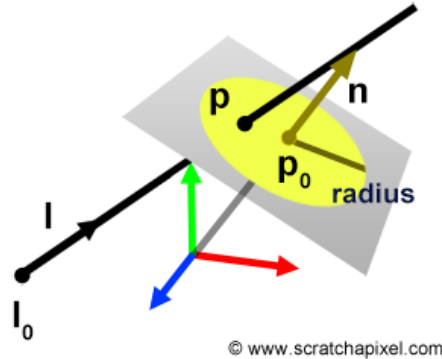


图 1: 光线和圆相交

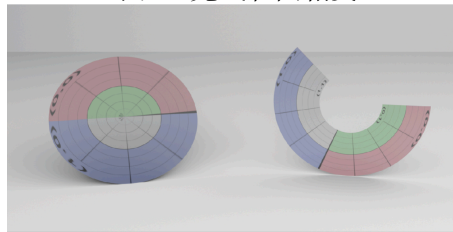


图 2: 两个圆环完整的和残缺的

### 3.2 求交

注意我们下文提到的 ray 都是转换到局部空间的 ray。在局部空间光线与圆环的求交就十分简单了, 可以通过联立方程计算也可以通过几何信息计算, 这里使用后者。我们先考虑光线和  $z=0$  这个平面的相交。首先需要检验光线方向  $z$  分量是否为 0。若不为 0, 计算光线到达该平面时的时间  $t = \frac{(0 - \text{ray.origin.z})}{\text{ray.direction.z}}$  我们可以使用 ray.at 函数来得到交点坐标, 在得到光线与平面的交点后, 我们需要检验以下几点:

- 我们需要考虑交点是在当前光线的范围内, 即判断  $t$  与光线的  $t_{\text{Far}}$  和  $t_{\text{Near}}$  的大小关系
- 交点是否在圆环半径内, 我们需要计算交点与圆心的距离。判断该距离和圆环的内外半径的大小关系
- 交点与圆心的角度是否符合要求, 需要计算交点和圆心所成夹角, 判断夹角和  $\text{phiMax}$  的大小关系

对于交点  $p$  与圆心的夹角  $\text{phi}$ , 我们有一个简单的几何关系  $\tan \text{phi} = \frac{p.y}{p.x}$ 。如果能通过以上检验, 那么就说明光线和物体我们的圆环是有交点的。我们还需要填

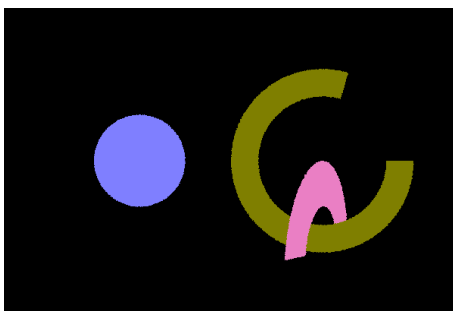
充 uv 坐标即纹理坐标。纹理坐标是一个二元组 (u,v)，详细介绍可以看 lab0。这里对于圆环我们计算 uv 可以用 u 表示交点在角度上的偏移，v 表示在半径上的偏移。 $u = \frac{\phi}{\phi_{\text{Max}}}$ ， $v = \frac{\text{radius} - \text{minRadius}}{\text{maxRadius} - \text{minRadius}}$  这样给定 uv 坐标，我们就可以在圆环上唯一的确定一个点。

### 3.3 填充求交信息

需要填充的有相交点位置和法线，注意使用 transform 类的 toWorld 将信息转换到世界空间。

### 3.4 验证你的求交结果

运行场景 lab1-test/lab1-test-disk 如果你的实现没有问题，那么你将得到以下图片结果。



## 4. 圆柱与光线的求交

### 4.1 moer-lite 中的圆柱

同圆环一样，我们一样在局部空间对圆柱进行求交。我们认为在局部空间内，圆柱底面是以原点为圆心，法线为 z 轴，半径为 r 的圆。圆柱高度为 h。因此，圆柱只需要两项来定义，radius 和 height。同圆环一样，我们为了能够渲染残缺圆柱，我们引入了 phiMax 表示圆柱最大角度。注意，moer-lite 中的圆柱只包含侧面。

```
1 class cylinder{
2     float height;
3     float radius;
4     float phiMax;
5 };
```

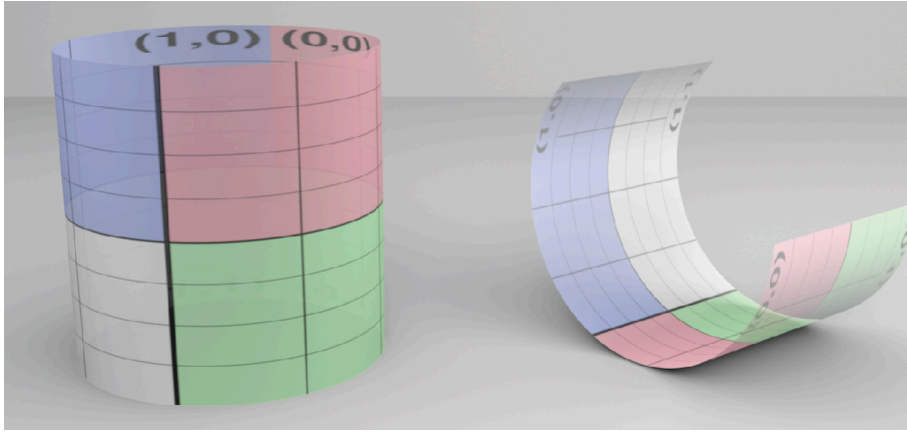


图 3: 两个圆柱完整的和残缺的

## 4.2 求交

在局部空间计算光线与圆柱的相交我们选择使用联立方程组来解决。感兴趣的同学可以考虑从几何上入手。

考虑无限高的圆柱，其方程为： $x^2 + y^2 = r^2$ 。而我们的光线方程为  $p = ray.direction * t + ray.origin$ 。联立二者，我们得到：

$$(ray.origin.x + ray.direction.x * t)^2 + (ray.origin.y + ray.direction.y * t)^2 = r^2$$

这是一个简单的一元二次方程组，我们在 Function.h 文件里面提供了一个简单的辅助函数求解。

```
11 inline bool Quadratic(float A, float B, float C, float *t0, float *t1);
```

A,B,C 分别为对应系数， $t_0, t_1$  为方程的解。 $t_0 < t_1$  但注意，我们这里求出的解是默认光线是无限长的直线，圆柱也是无限高的。得到交点后，我们需要检验  $t$ 。

- 我们需要考虑交点是在当前光线的范围内，即判断  $t$  与光线的  $tFar$  和  $tNear$  的大小关系
- 交点是否在圆环内，交点的  $z$  坐标是否在圆柱高度范围内
- 交点与圆柱对应面的圆心的角度是否符合要求，需要计算交点和圆心所成夹角，判断夹角和  $phiMax$  的大小关系

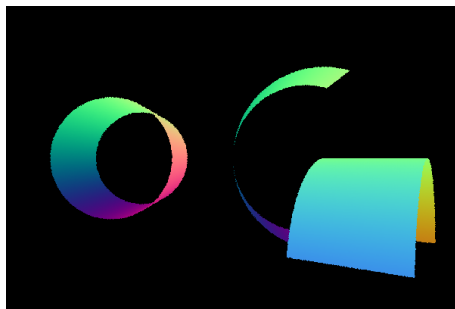
如果  $t$  可以通过以上检验，那么我们的光线和圆柱就有交点。要注意的一点是如果方程有两个解，并且都符合要求，我们需要选择最近的即  $t_0$ 。通过  $t$  得到交点坐标，我们可以通过  $u = \frac{phi}{phiMax}$   $v = \frac{p.z}{height}$  计算纹理坐标。

### 4.3 填充求交信息

需要填充的有相交点位置和法线，注意使用 transform 类的 toWorld 将信息转换到世界空间。交点处的位置可以根据 uv 坐标计算，交点处的法线沿该点半径方向。 $n = (\cos(phi), \sin(phi), 0)$ 。

### 4.4 验证你的求交结果

运行场景 lab1-test/lab1-test-cylinder 如果你的实现没有问题，那么你将得到以下结果。



## 5. 圆锥与光线的求交

### 5.1 moer-lite 中的圆锥

同样的，我们一样在局部空间对圆锥进行求交。我们认为在局部空间内，圆锥底面是以原点为圆心，法线为 z 轴，半径为 r 的圆。圆锥高度为 h。因此，圆锥只需要两项来定义 radius 和 height。同前面两个物体一样，我们为了能够渲染残缺圆锥，我们引入了 phiMax 表示圆锥最大角度。为了计算方便，我们记录圆锥夹角的 cos 值。注意，moer-lite 中的圆锥只包含侧面。

```
1 class cylinder{
2     float height;
3     float radius;
4     float phiMax;
5     float cosTheta;
6 };
```

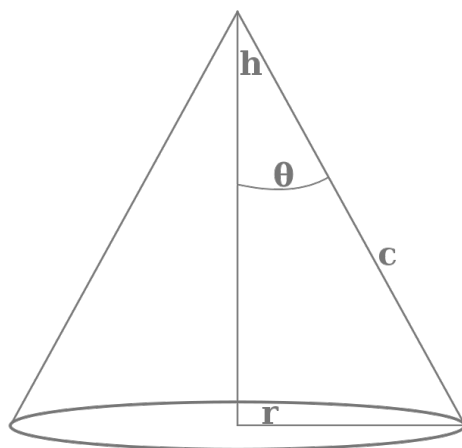


图 4: 圆锥示意图

## 5.2 求交

在局部空间计算光线与圆锥的相交我们选择使用联立方程组来解决。感兴趣的同学可以考虑从几何上入手。

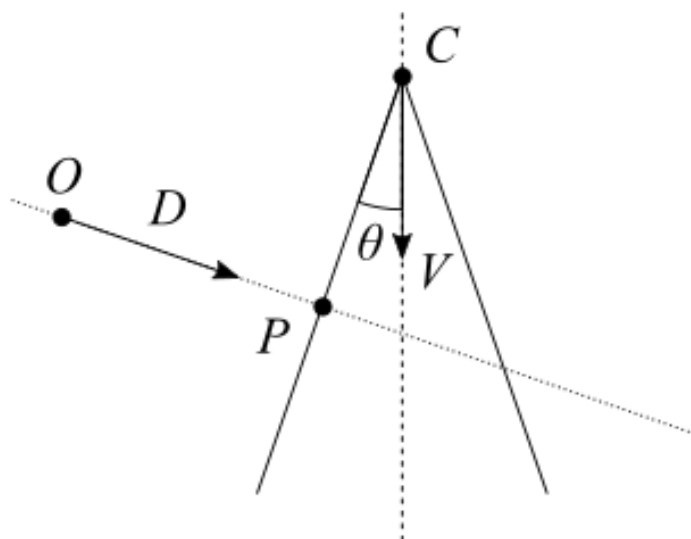


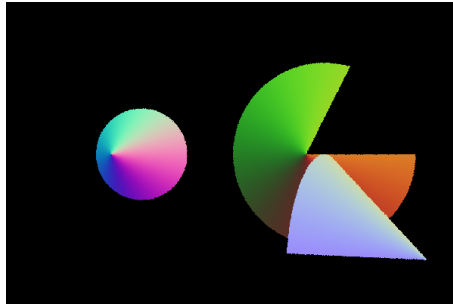
图 5: cone-ray-intersection

圆锥上的任意点  $P$  都满足  $(P - C) \cdot \vec{V} = |P - C| \cos \theta$  同时光线的方程为  $P = O + t \cdot \vec{D}$  联立并带入我们得到方程:  $t^2(\vec{D} \cdot \vec{D})(\vec{V} \cdot \vec{V}) - 2t(\vec{D} \cdot \vec{V})(O - C) \cdot \vec{V} +$



### 5.4 验证你的求交结果

运行场景 `lab1-test/lab1-test-cone1` 如果你的实现没有问题，那么你将得到以下图片结果。



## 6. 加速结构介绍

本部分为选做部分。即使不完成也不会影响分数。如果你选择做，只需要选一种加速结构完成即可

### 6.1 加速结构简介

在这次作业中，我们将实现用来加速光线与场景中物体求交的数据结构。在光线追踪渲染器中，无论是寻找着色点或者是判断光线与物体的遮挡关系等等，都需要我们计算光线与物体的相交。一个朴素的想法是，对于一条给定的光线，我们遍历场景中的每个物体，计算光线是否与物体相交来完成这一任务。这样的想法对于 `cornell-box` 这样的场景是可行的。但是许多场景中可能有几十个或者更多物体。有的几何物体例如 `TriangleMesh` 可能由几十万个三角形面片组成，毛发可能由几十万条曲线组成。暴力遍历的方式的时间成本是难以接受的。我们可以根据物体在三维空间内的几何分布和物体的层次结构来加速这一过程。



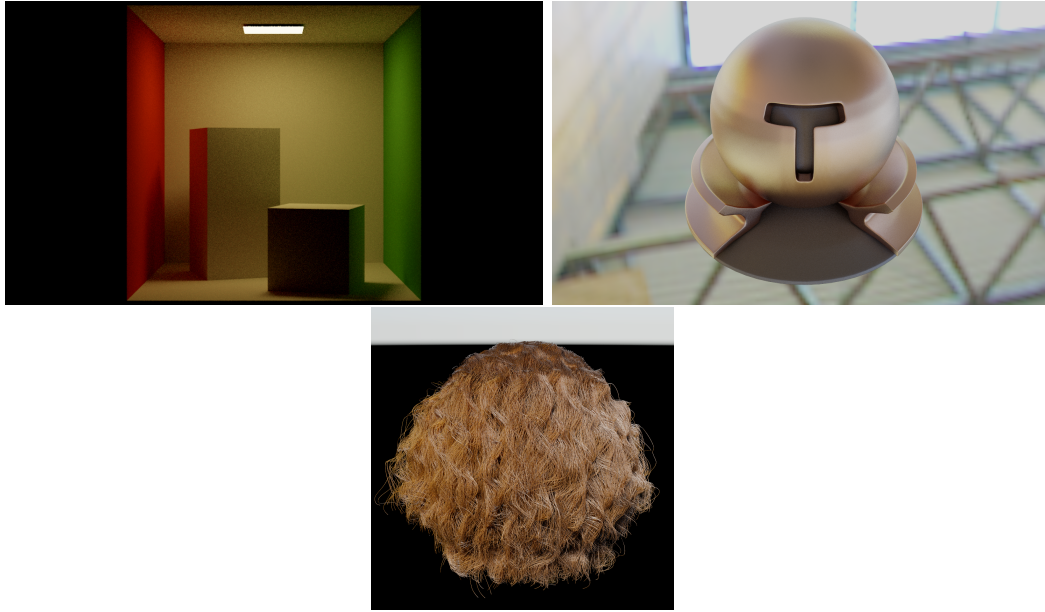


图 7: cornell-box test-ball curly-hair

## 6.2 BoundingBox

要加速求交，显然需要从物体的几何结构入手。然而不同几何结构性质不同，很难统一考虑。我们为物体引入包围盒这一概念。光线与包围盒的求交是非常容易计算的。容易想到，如果光线与包围盒不相交，那么光线与物体一定不相交。我们可以利用这一点进行加速。在 moer-lite 中，BoundingBox 这个类成员很简单，只需要记录 box 的最小点和最大点就可以。

```

1  class AABB{
2      Point3f pMin, pMax;
3      bool RayIntersect(const Ray &ray, float *tMin = nullptr,
4                          float *tMax = nullptr) const;
5      ...
6  }
7

```

这里挑出 BoundingBox 的一个方法进行说明 (RayIntersect)。要计算光线是否与包围盒相交，我们遍历 xyz 三个轴对应的两个面，计算光线在对应两个面进出包围盒的时间，更新光线与包围盒相交 最小相交时间与最大相交时间。光线与包围盒没有相交当且仅当最小相交时间大于最大相交时间。

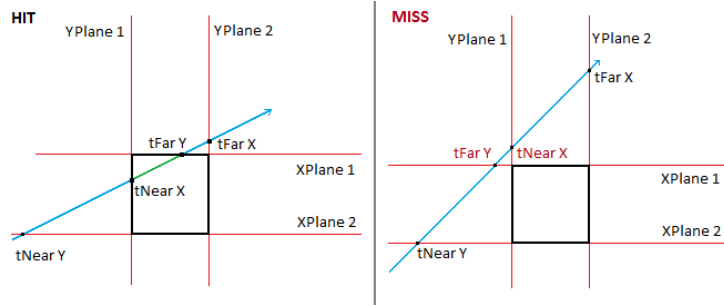


图 8: ray intersect with AABB

BoudingBox 类的余下成员方法可以在 BoudingBox 类文件里面查看。

### 6.3 加速结构接口

在 moer-lite 中，加速结构主要接口如下

```

12  /** 所有空间加速结构需要实现的接口
13  class Acceleration {
14      virtual bool rayIntersect(Ray &ray, int *geomID, int *primID, float *u,
15      float *v) const = 0;
16      virtual void build() = 0;
17      std::optional<Intersection> rayIntersect(Ray &ray) const {
18          int primID, geomID = -1;
19          float u, v;
20          bool hit = rayIntersect(ray, &geomID, &primID, &u, &v);
21          if (!hit)
22              return std::nullopt;
23          Intersection its;
24          shapes[geomID]->fillIntersection(ray.tFar, primID, u, v, &its);
25          return std::make_optional(its);
26      }
27      ...
28  }
29  
```

我们首先来思考一个问题，除了判断光线是否与物体相交我们还需要记录哪些信息。我们需要记录着色点的法线，纹理信息等。这些做法是不需要在求交时做的，因为无法判断当前求交的物体是不是最近的那个物体。所以在求交过程中，我们只需要记录最终求交物体在场景中的 shapeId 和 primId。二者分别是相交物体在场景中的唯一标识 和具体和物体的哪个图元（一般为 0，除非像 triangleMesh 这样的多图元几何结构）相交。

有了这样的思路，理解加速结构的接口就很简单了。在非虚函数 rayIntersect 方法中，我们调用每个加速结构自己实现的求交方法，记录 shapeId 和 primId。然后找到对应的物体，只要我们 给它提供 primId 和 uv 坐标，那么它就一定能帮我们填充好 Intersection 信息。

Build 函数则是每个加速结构需要在场景构建的过程中调用的方法。每种加速结

构构建的方法不一样，所以设置为虚函数，也是我们这次作业中需要实现的。

在 moer-lite 中，主场景拥有加速结构，每个 mesh 都会拥有自己的加速结构。同学们不妨想一想，这样的设计比将 mesh 分成独立的小三角形，然后只用一个主场景的一个加速结构 完成场景求交有什么优势。

## 7. Octree (选做)

### 7.1 Octree 介绍

本节进入我们本次作业的第一个加速结构-Octree (八叉树) 介绍。这种加速结构通过对空间进行划分来完成求交加速。八叉树它将三维空间分割成 8 个同等大小的子空间，为什么是分成八份呢，因为我们空间是三维的，每一个轴分成两份。如果我们在二维空间构建，我们就只需要使用四叉树。我们递归地将每个子空间分割成更小的子空间，每个子空间包含和它有相交的物体 id。直到每个子空间中包含的物体数小于某个阈值或到达了最大深度，在求交时，如果光线和当前节点对应的空间没有相交的话，我们就可以直接跳过当前节点，这样的剪枝大大提高了光线求交的效率。如果光线和当前节点有相交的话，就可以递归遍历当前节点 的子节点完成求交。

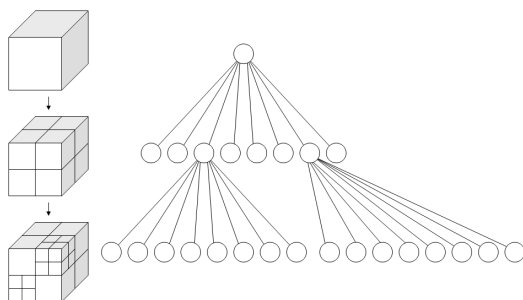


图 9: octree

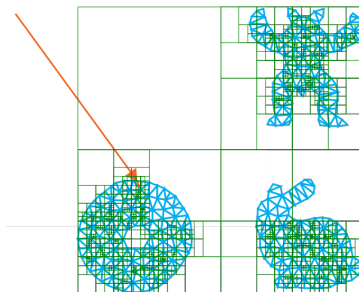


图 10: octree

## 7.2 Octree 节点结构

有了这样的求交思路，我们就可以设计出 OctNode 的结构。对于非叶子结点，我们需要存储 8 个子节点的指针用来递归遍历，以及节点的包围盒用来判断光线有没有可能和当前节点下的物体相交。对于叶子结点，我们需要记录叶子结点的存储的物体 Id 这里给出一个参考。

```

1 struct Octree::OctreeNode {
2     BoundingBox box;
3     std::unique_ptr < std::vector <int > > indexBuffer;
4     OctreeNode * subNodes[nSub];
5     ...
6 }
7

```

## 7.3 Octree 构建

在有了结点结构后，我们需要考虑如何构建八叉树。根据我们上文提到的节点结构，构建时我们需要节点的包围盒以及当前节点需要处理的索引数组。如果递归深度到达最大值或者当前节点数小于一定值时，当前节点作为叶子节点。否则我们将当前节点八等分。用八个子节点表示。子节点的包围盒通过空间划分得到。子节点的索引数组通过寻找和子节点包围盒有重叠区域的物体得到。这里给出伪代码。

```

1 Node recurseveBuild(box,indexBuffers,depth):
2     if(indexBuffers.size()<MaxLeafSize or depth > maxDepth)
3         reutrn node(indexBuffers)
4     else :
5         Node node(box)
6         subBoxes = getSubBoxes(box)
7         vector<vector<int>> subBuffers(8)
8         for(i = 0; i<8 ;i++)
9             for(index : indexBuffers)
10                 if(shapes[index].getAABB().overlaps(subBoxes[i]))
11                     subBuffers[i].push(index)
12         for(int i = 0;i<8;i++)
13             node.subNodes[i] = recurseveBuild(subBoxes[i],subBuffers[i])
14         return node
15

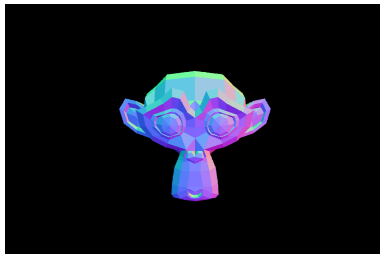
```

## 7.4 Octree 求交

八叉树求交逻辑在八叉树介绍时我们就已经提到过了。主要是一个递归过程，通过节点的包围盒对与光线是否相交进行剪枝来进行加速。对于非叶子节点，首先判断当前空间是否和光线相交，如果不相交可以直接返回，否则遍历每个子节点。遍历到叶子节点时我们遍历叶子节点下的所有物体进行求交即可。

## 7.5 验证

运行场景 `lab1-test/lab1-test-octree` 如果你的 octree 实现没有问题，那么你将在较短时间内（不超过 5s）完成渲染得到 `monkey.obj` 的法线图



# 8. BVH (选做)

## 8.1 BVH 介绍

本节进入我们本次作业的二个加速结构-BVH (Bounding Volume Hierarchies) 介绍。与 Octree 对空间进行划分不同, BVH 通过对物体进行划分来完成加速。BVH 的核心思想就是用体积略大而几何特征简单的包围盒来近似描述复杂的几何对象, 并且这种包围盒是嵌套的, 我们只需要对包围盒进行进一步的相交测试, 就可以越来越逼近实际对象。在求交时, 如果光线和当前节点对应的包围盒没有相交的话, 我们就可以直接跳过当前节点, 这样的剪枝提高了光线求交的效率。如果光线和当前节点有相交的话, 就可以递归遍历当前节点的左右子节点完成求交。

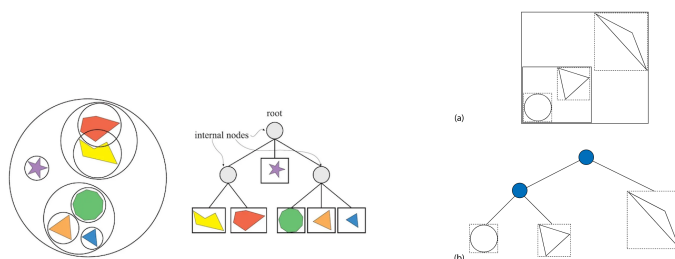


图 11: bvh

## 8.2 BVH 节点结构

同样的, bvh 也是树形结构。分为叶子结点和非叶子结点, 叶子结点存储物体索引, 非叶子结点存储节点包围盒以及左右节点索引。(该结构仅为讲解方便作示例, 实际实现不必拘泥)

```
1 struct BVH::BVHNode {
2     BVHNode * left;
3     BVHNode * right;
```

```

4   BoundingBox box;
5   int firstShapeOffset;
6   int nShape = 0;
7   int splitAxis;
8 }
9

```

其中 `firstShapeOffset` 和 `nShape` 是为叶子结点存结点索引使用（代表第一个物体的索引和叶子结点存了多少物体）。`splitAxis` 代表非叶子结点的分割轴（在后续求交部分会用到）

### 8.3 BVH 构建

在有了结点结构后，我们需要考虑如何构建 BVH。对于非叶子节点在最开始，我们需要处理整个场景的物体索引。现在我们要想一种分割方式，将这一组物体分成两组，再交给左右子节点分别构建。这种分割方式要保证我们在相交时能够尽量少的进行相交测试。分割方式很大程度上影响 BVH 的性能。一种常见的方式是按中点分割，将所有物体的中心在某个坐标轴上排列，按照中心在该坐标轴的大小均匀分成两部分。

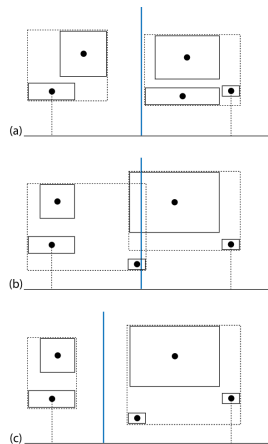


图 12: 中点分割

这种方式简单，但分割效果不一定好。如上图所示。在 (a) 中所示的情况，根据质心沿所选轴线的中点来进行分割是有效的。由此产生的两个形状组的包围盒用虚线表示。然而，对于像 (b) 中所示的这种分布，中点选择是欠优的，两个中间包围盒的相互重叠会导致性能下降。因此，(c) 中选择一个不同的分割平面（绿色），导致得到的包围盒更小且完全不重叠，从而在渲染时获得更好的性能。

感兴趣的同学可以考虑实现 SAH，并且比较下 SAH 和中点分割的性能差异。SAH 会考虑不同分割方式的求交成本。 [SAH 分割参考](#)

这里我们只实现中点分割就可以。按照物体中心分割，物体中心可以认为是物体

的包围盒中心。在 bvh 构造函数接受的索引数组小于我们设定的叶子结点最大物体数时 我们就可以结束递归，返回一个叶子结点。

```

30  BVHNode * BVH::recursiveBuild(std::vector < BVHShapeInfo > & shapeInfos, int l, int r,
31                                std::vector < std::shared_ptr < Shape>> & orderedShapes) {
32  if ( r-l <= LeafMaxSize ) {
33      node->firstShapeOffset = orderedShapes.size();
34      node->nShape = nShapes;
35      for ( int i = l ; i < r ; ++ i ) {
36          orderedShapes.push_back(shapes[shapeInfos[i].id]);
37      }
38      return node;
39  }
40  \... handle no-left case
41  }

```

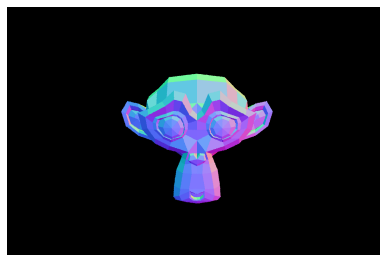
叶子结点需要存储节点内所有物体的索引，我们可以像八叉树一样存一个索引数组。但对于 bvh 来说，在分割构建时我们会对物体进行排序，此时叶子结点接受的序列列表是有序的，我们存储第一个物体的索引和物体个数就可以。每次初始化叶子结点，我们都会向 orderedShapes 里存储排好的物体。在 bvh 构建完成后，我们会将 shapes 重置为 orderedShapes，以保证叶子结点中存储的索引和 bvh 中物体列表可以对应。

## 8.4 BVH 求交

BVH 求交逻辑非常简单。给定光线，我们首先判断节点是否为叶子节点，如果是叶子节点，我们依次遍历节点物体即可。对于非叶子节点，我们之前就递归调用左右子节点的求交函数即可。当然求交也可以不使用递归，用一个数组存储待处理的节点迭代处理是更好的选择。这里我们要注意的一点是，我们是先调用左子节点还是右子节点呢。不同的调用顺序显然会影响求交效率。例如光线因为和左子节点相交的而更新了 tFar 而不会与右子节点的包围盒相交。这种情况下先调用左子节点的求交就可以减少求交计算次数。还记得我们的 splitAxis 这个属性吗。我们在某一个轴上将物体坐标从小到大排列。我们可以根据光线在这个轴的方向上选择调用顺序。如果光线在这个方向上的分量大于 0，我们就先调用左子节点。

## 8.5 验证

运行场景 `lab1-test/lab1-test-bvh` 如果你的 bvh 实现没有问题，那么你将在较短时间内（不超过 5s）完成渲染 得到 monkey.obj 的法线图





## 9. 景深

目前 moer-lite 提供了 pinhole（即针孔相机）的实现，即类似于小孔成像。而真实的相机通常可以调节**光圈**和**焦距**等参数，以呈现景深（Depth of field）的效果。



图 13: PBRT-v4景深效果示意

现在我们提供的针孔相机，其实现逻辑如下：

```

1 Ray PinholeCamera::sampleRay(const CameraSample &sample, Vector2f NDC) const {
2     // 目前针孔相机采样光线的逻辑：
3     // 这里的 x, y, z是在计算，当前像素上的采样点的坐标位置
4     float x = (NDC[0] - 0.5f) * film->size[0] + sample.xy[0],
5           y = (0.5f - NDC[1]) * film->size[1] + sample.xy[1];
6
7     float tanHalfFov = fm::tan(verticalFov * 0.5f);
8     float z = -film->size[1] * 0.5f / tanHalfFov;
9
10    // 以针孔位置为原点，计算当前光线的方向
11    Vector3f direction = Vector3f{x, y, z};
12
13    // 将光线方向、起点转换到世界坐标系下
14    direction = normalize(transform.toWorld(direction));
15    Point3f origin = transform.toWorld(Point3f(0));
16
17    // TODO 如果要实现动态模糊，那么此处需要根据sample中的time采样一个时间
18    return Ray(origin, direction, tNear, tFar, timeStart);
19 }

```



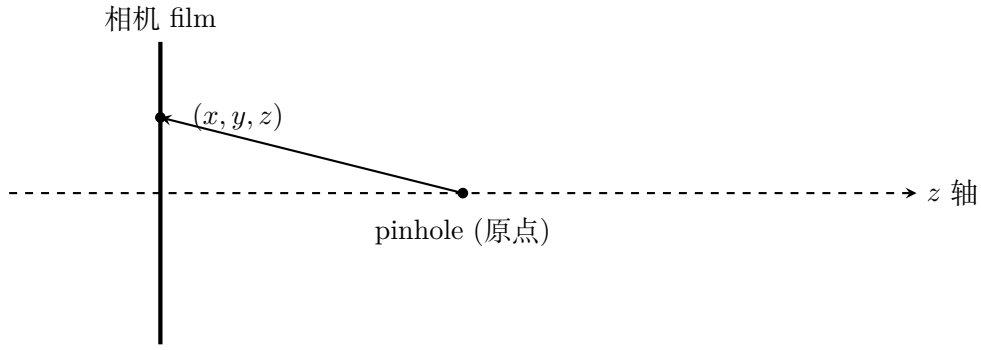


图 14: 目前针孔相机的示意图, 我们根据 NDC 和像素内 jitter, 确定 film 上采样点的位置

## 9.1 实现透镜相机

```

1
2 // ThinLens.h 透镜相机的定义
3 class ThinLensCamera : public PerspectiveCamera {
4
5 public:
6     ThinLensCamera() = delete;
7
8     ThinLensCamera(const Json& json);
9
10    virtual Ray sampleRay(const CameraSample& sample, Vector2f NDC) const override;
11
12    virtual Ray sampleRayDifferentials(const CameraSample& sample, Vector2f NDC) const override;
13
14 private:
15     float lensRadius, focalDistance;
16 };
17
18 // ThinLens.cpp 透镜相机的实现, 你需要完成的部分
19
20 // 参数的解析和读取已经实现
21 ThinLensCamera::ThinLensCamera(const Json& json) : PerspectiveCamera(json) {
22     lensRadius    = json["lensRadius"].get<float>();
23     focalDistance = json["focalDistance"].get<float>();
24 }
25
26 Ray ThinLensCamera::sampleRay(const CameraSample& sample, Vector2f NDC) const
27 {
28     // TODO
29 }
30
31 Ray ThinLensCamera::sampleRayDifferentials(const CameraSample& sample, Vector2f NDC) const
32 {
33     // TODO
34 }

```

ThinLensCamera 同样继承了透视相机的接口, 我们增添了两个新参数, 其中 lensRadius 控制透镜的半径大小, focalDistance 控制焦平面的距离。

对于透镜相机模型, 我们假设 只有位于焦平面上的点, 会在经过透镜后汇聚到

成像平面上的一个点，如下图所示。

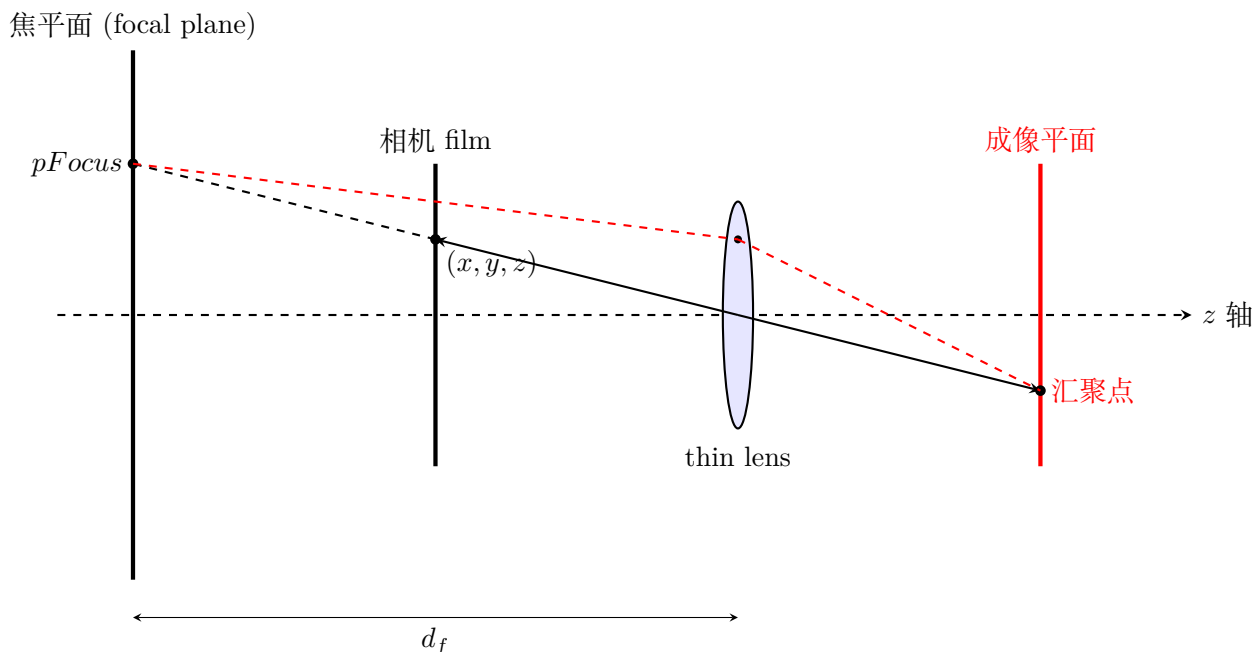


图 15: 透镜相机的示意图，其中  $d_f$  为焦距

因此，你需要实现的逻辑为：

- 根据 NDC 坐标与像素内 jitter，确认  $pFocus$  的位置
- 随机的在  $thinlens$  上采样一个点，随机数使用 `CameraSample` 中的 `lens`，你需要完成 **在圆盘上，以均匀概率进行随机采样**的逻辑
- 以  $thinlens$  上的采样点为起点，朝  $pFocus$  方向即为光线的方向（不要忘记转换为世界坐标）

实现完毕后，可以将 `examples/cornell-box-dof` 作为参数传入进行渲染，你将能够得到具有景深效果的渲染图，其结果如下：

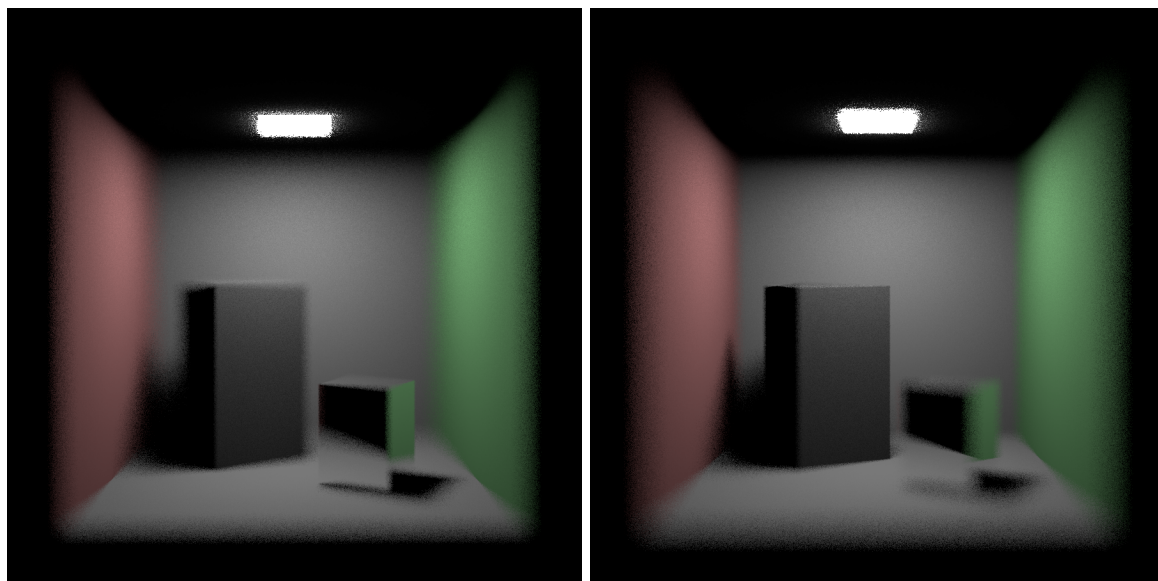


图 16: 实现后, 分别将相机的 focalDistance 参数设置为 5.7 和 6.5 的结果, 可以看到前者聚焦在靠前的长方体上, 而后者聚焦在后方的长方体上。

## 10. 勘误 & 补充

Update : 2026-4-13, 非常感谢陈俊杰同学指出以下两点问题:

- 对于**法线**从局部坐标转换至世界坐标的操作, 更为严谨的实现方式为 [PBRT-v4 Applying Transformations](#)一节中使用的形式, 即使用 **toWorld** 的**逆的转置**, 这种形式能够更好的处理不规则的 scale (即沿不同方向缩放不同系数时, 引起的法线变化)。由于目前提供的测试文件中, 只有 **cube** 使用了这种 scale, 但 **cube** 的 **normal** 在变化时只使用了 **rotate** (故不影响 **cube** 法线变化的正确性)。
- 出于遗留问题, 在 Transform 类中的 inverseRay 方法里, 我们的实现为

```
1 // Transform.cpp : 89 line
2 Ray Transform::inverseRay(const Ray &ray) const {
3     Point3f origin = ray.origin;
4     Vector3f direction = ray.direction;
5
6     vecmat::vec4f o{origin[0], origin[1], origin[2], 1.f},
7         d{direction[0], direction[1], direction[2], 0.f};
8
9     o = invRotate * invTranslate * o;
10    d = invRotate * invTranslate * d;
11
12    o /= o[3];
13    origin = Point3f{o[0], o[1], o[2]};
14    direction = Vector3f{d[0], d[1], d[2]};
15    return {origin,direction,ray.tNear,ray.tFar};
16 }
17
```

可以看到里面对光线的起点、方向只使用了 rotate 和 translate 的逆变换。其原因在于我们在实现 Cube 类时, 手动处理了 Cube 的缩放

```
1 // Cube.cpp : 6 line
2 Cube::Cube(const Json &json) : Shape(json) {
3     // Cube默认是中心位于(0, 0, 0)处, 棱长为2的AABB, 其形状可以通过transform调整
4     boxMin = Point3f{-1.f, -1.f, -1.f};
5     boxMax = Point3f{1.f, 1.f, 1.f};
6
7     // 构建AABB
8     for (int i = 0; i < 8; ++i) {
9         Point3f p;
10        p[0] = (i & 0b100) ? boxMax[0] : boxMin[0];
11        p[1] = (i & 0b010) ? boxMax[1] : boxMin[1];
12        p[2] = (i & 0b001) ? boxMax[2] : boxMin[2];
13        p = transform.toWorld(p);
14
15        for (int j = 0; j < 3; ++j) {
16            boundingBox.Expand(p);
17        }
18    }
19 }
```

```
19
20 // 在计算时，所有计算都是在局部坐标系内完成的，因此这里只对boxMin和boxMax做scale操作
21 Matrix4f scale = transform.scale; //!这里
22 vecmat::vec4f min{boxMin[0], boxMin[1], boxMin[2], 1.f},
23               max{boxMax[0], boxMax[1], boxMax[2], 1.f};
24 min = scale * min, max = scale * max;
25 min /= min[3], max /= max[3];
26 boxMin = Point3f{min[0], min[1], min[2]};
27 boxMax = Point3f{max[0], max[1], max[2]};
28 }
29
```

这是一个遗留的历史问题，会导致我们需要手动处理所有 shape 的 scale 缩放。为此我们去掉了 lab1-test-disk 测试场景中，最后一个 disk 的 scale。因此，目前除了 Cube（框架中已经处理），不再涉及 scale 的 transform，规避了这个遗留问题。